A. Proposed Approach

Our approach relies on a modular monolith architecture. This balances development velocity with strong isolation, avoiding the overhead of managing distributed microservices. We chose Keycloak (v26.8.0) as our Identity Provider to handle authentication, OIDC flows, and token lifecycles. Building a custom IdP is risky, and Keycloak provides industry-standard security out of the box.

For authorization, we use Casbin as our policy engine. While Keycloak has built-in authorization, it proved too rigid for our dynamic, tenant-specific, and relationship-based rules (like "instructors can only grade their assigned courses"). Casbin lets us define clear Role-Based Access Control (RBAC) rules, while our application database (PostgreSQL v16.15) acts as the source of truth for dynamic relationships and active tenant memberships.

B. System Design

The system architecture includes the following major components, all deployed via Docker Compose for reproducibility:

- **Clients:** Lightweight web and mobile-style browser interfaces. They use Authorization Code Flow with PKCE. The clients never handle user passwords directly.
- **Identity Provider (Keycloak):** Manages credentials, enterprise federation (OIDC brokering), and issues JSON Web Tokens (JWTs).
- **Backend API (Fastify):** A Node.js backend that validates JWT signatures, enforces uncached introspection, and maps the provider subject to our internal users.
- **Database (PostgreSQL):** Stores application state including organizations, memberships, learning resources, and audit logs. It enforces strict data integrity using unique constraints, quotas, and composite foreign keys to prevent cross-tenant data leaks.

Workflow: When a user accesses the platform, they authenticate via Keycloak and receive an access token. The client sends this token along with an `X-Organization-ID` header to our API. The backend strictly validates the token and checks if the user has an active membership in the requested tenant. Casbin then evaluates the user's role against the requested resource. The API executes the action and commits an audit log within the same database transaction.

C. Technical Feasibility

Tools and Infrastructure:

- Node.js (v24.21.0), Fastify (v5.12.5)
- PostgreSQL (v16.15) with Kysely for migrations
- Keycloak (v26.8.0)
- Casbin (v5.51.1)

- Docker and Docker Compose
- Vitest and Playwright for security and browser testing

Challenges and Mitigation: One of the major challenges we addressed this week was ensuring the safety of our introspection logic. We added strict environment validation so that introspection cannot be accidentally disabled in production (failing closed). We also verified that local PostgreSQL transactions handle application provisioning atomically, keeping it isolated from any external IdP modifications. Another insight was that a password reset does not automatically revoke existing Keycloak access sessions, so we documented that incident response must include an explicit `logout-all`.

D. Implementation Plan

Completed Work (Weeks 1-3):

- Full Docker Compose infrastructure setup with private databases.
- Integration of Keycloak for OIDC PKCE login and enterprise federation.
- Fastify backend with robust JWT validation, strict tenant context middleware, and database-backed Casbin authorization.
- Addressed all P0/P1 mentor feedback: removed hardcoded configurations, enforced introspection safety, and added comprehensive tenant boundary tests.
- Created a script (`verify-test-independence.mjs`) to prove that bypassing our security guards correctly causes test failures.
- Implemented and verified 159 automated tests covering authorization, tenant isolation, and live session lifecycles.
- Verified that a completely clean clone can stand up the entire environment and pass all tests automatically.

Ongoing and Pending Tasks:

- *Ongoing:* Formalizing our evaluation protocol and freezing the core security baseline.
- *Pending:* Implementing optional extensions, such as application machine identities and user delegation.
- *Pending:* Finalizing the performance optimization experiment (comparing sequential lookups vs. composite indexing/batched queries against a frozen baseline).

Expected Outcomes: The expected outcome is a fully functional, highly secure, and well-documented prototype demonstrating strong tenant isolation and declarative authorization. The core framework is now stable, and we are preparing for final optimizations and the evaluation phase.